
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 12

Estruturas de Dados em C

Seção Especial

Construindo seu próprio compilador
(Exercícios 12.26 – 12.30)

Reflexão final do capítulo

Contents

1	Visão Geral do Projeto do Compilador	2
2	Exercício 12.26 – Parser de Simple	4
3	Exercício 12.27 – Geração de Código SML	6
4	Exercícios 12.28 a 12.30 – Extensões	7

Nota Introdutória

Atenção

O Capítulo 12 do livro **não tem seção “Fazendo a Diferença”** tradicional. Em seu lugar, encontra-se a **Seção Especial sobre Construção de Compilador** (exercícios 12.26 a 12.30), que constitui um projeto de várias semanas integrando *todos* os conceitos vistos até aqui.

Este documento apresenta:

- Visão geral dos exercícios 12.26–12.30 (texto integral disponível em PDF separado no site da Deitel).
- Esqueletos de implementação dos componentes principais.
- Reflexão final consolidando todo o conhecimento do livro até o Cap. 12.

1. Visão Geral do Projeto do Compilador

Contexto integrado

O projeto integra três trabalhos anteriores:

- **Cap. 7:** simulador Simpletron e linguagem SML (exercícios 7.27–7.29).
- **Cap. 11:** leitura/escrita de programas em arquivos (Ex. 11.17).
- **Cap. 12:** estruturas de dados (pilhas, filas, listas, árvores) que o compilador precisa para processar código.

O **objetivo**: criar um compilador que traduza programas escritos em uma linguagem de alto nível chamada **Simple** (variante didática de BASIC) para Simpletron Machine Language (SML), que então executa no simulador do Cap. 7.

A linguagem Simple

Resposta detalhada

Comandos da linguagem Simple (sintaxe estilo BASIC):

```
1 10 rem este e um comentario
2 20 input x          /* le inteiro do teclado e
   armazena em x */
3 30 let y = x * 2     /* atribuicao com expressao */
4 40 if y > 100 goto 60
5 50 print y
6 60 goto 20
7 70 end
```

Características:

- Cada linha começa com um **número de linha** (rótulo).
- Tipos: apenas inteiros (mapeados a palavras SML).
- Variáveis: identificadores simples (a, x, i, ...).
- Operadores: +, -, *, /, %, comparadores <, >, <=, >=, ==, !=.

- Controle: goto, if-goto.

Exemplo de programa Simple

Resposta detalhada

Calcular a média de inteiros lidos (sentinela = -1):

```
1 10  rem programa de media com sentinela
2 20  rem total acumulado em t, contador em c
3 30  let t = 0
4 40  let c = 0
5 50  input n
6 60  if n == -1 goto 110
7 70  let t = t + n
8 80  let c = c + 1
9 90  goto 50
10 100 rem calcula e imprime a media
11 110 if c == 0 goto 140
12 120 let m = t / c
13 130 print m
14 140 end
```

2. Exercício 12.26 – Parser de Simple

Resposta detalhada

Etapas de um parser:

- Análise léxica (*tokenization*)**: separar a entrada em tokens (palavras-chave, identificadores, números, operadores).
- Análise sintática**: verificar se a sequência de tokens segue a gramática da linguagem.
- Geração de tabela de símbolos**: mapear identificadores e rótulos para posições em memória.

Esqueleto da tabela de símbolos (struct do Cap. 10):

```
1 typedef struct {
2     int    simbolo;      /* numero de linha ou ASCII de
                           variavel */
3     char  tipo;          /* 'C' = constante, 'L' = linha,
                           'V' = variavel */
4     int    posicao;       /* posicao em memoria SML (0-99)
                           */
5 } Simbolo;
6
7 Simbolo tabela[ 100 ];
8 int tamanhoTabela = 0;
```

Parser principal (esqueleto):

```
1 typedef struct linhaSimple {
2     int    numeroLinha;
3     char   comando[ 100 ];
4     int    posicaoSML;    /* preenchido na
                           primeira passada */
5     struct linhaSimple *prox;
6 } LinhaSimple;
7
8 LinhaSimple* parseArquivo( const char *nomeArq )
9 {
10     FILE *fp = fopen( nomeArq, "r" );
11     LinhaSimple *lista = NULL, *ultimo = NULL;
12     char buf[ 200 ];
13
14     while ( fgets( buf, sizeof( buf ), fp ) != NULL )
15     {
16         LinhaSimple *l = malloc( sizeof( LinhaSimple ) );
17         sscanf( buf, "%d %[^\\n]", &l->numeroLinha, l->comando );
18         l->posicaoSML = -1;
19         l->prox      = NULL;
20         if ( lista == NULL ) lista = l;
21         else              ultimo->prox = l;
22         ultimo = l;
```

```
22     }  
23     fclose( fp );  
24     return lista;  
25 }
```

3. Exercício 12.27 – Geração de Código SML

Resposta detalhada

Estratégia de duas passadas:

Primeira passada: percorrer todas as linhas Simple e:

- Atribuir uma posição SML para cada *linha* (rótulo).
- Atribuir uma posição SML para cada *variável* encontrada.
- Não resolver ainda os `goto` (eles podem apontar para frente).

Segunda passada: re-percorrer e gerar instruções SML reais, agora com todos os endereços conhecidos.

Mapeamento típico (Simple → SML):

Simple	SML gerado
<code>rem ...</code>	nada
<code>input x</code>	READ x
<code>print x</code>	WRITE x
<code>let x = expr</code>	avalia expr, STORE x
<code>goto NL</code>	BRANCH posSML(NL)
<code>if a == b goto NL</code>	LOAD a, SUBTRACT b, BRANCHZERO NL
<code>if a < b goto NL</code>	LOAD a, SUBTRACT b, BRANCHNEG NL
<code>end</code>	HALT

Tradução de expressões com pilha (Ex. 12.12!):

A expressão `let z = (a + b) * 2` converte-se em pós-fixado (`a b + 2 *`) e gera SML:

```

1  LOAD a
2  ADD b
3  STORE temp1
4  LOAD temp1
5  MULTIPLY const2  /* const2 = posicao da constante 2 */
6  STORE z

```

Aqui está a beleza: o Ex. 12.12 (conversão infixado → pós-fixado) e o Ex. 12.13 (avaliação pós-fixada) que pareciam exercícios isolados são, na verdade, **componentes essenciais** de um compilador real!

4. Exercícios 12.28 a 12.30 – Extensões

Resposta detalhada

12.28 – Melhoria do compilador: otimizações comuns:

- **Constant folding:** avaliar expressões constantes em tempo de compilação (`let x = 2 + 3 → let x = 5`).
- **Eliminação de código morto:** remover linhas inacessíveis.
- **Compartilhamento de constantes:** se a constante 2 aparece várias vezes, ela ocupa apenas uma posição SML.

12.29 – Recursos avançados:

- Comandos compostos com chaves (`{ ... }`).
- Estruturas `while-end` e `for-end`.
- Tipos adicionais (ponto flutuante, strings curtas).
- Funções definidas pelo usuário (precisaria expandir SML também).

12.30 – Testes integrados:

- a) Escrever programas Simple.
- b) Compilá-los com o compilador.
- c) Executá-los no Simpletron.
- d) Verificar saídas esperadas.

Este é o **ciclo completo** da computação: linguagem de alto nível → compilação → código de máquina → execução. Compreender esse ciclo é o que separa um programador básico de um cientista da computação.

Reflexão Final – O Encerramento do Núcleo de C

12 capítulos, uma jornada completa

Chegando ao Capítulo 12, você completou o **núcleo essencial de C**. Olhando para trás:

Capítulos 1–2 – Fundamentos:

- Computadores, hardware, software.
- Primeiros programas: `printf`, `scanf`, `if`.

Capítulos 3–5 – Programação Estruturada:

- `while`, `for`, `do-while`, `switch`.
- Operadores lógicos, atribuição composta.
- Funções, recursão, escopo, números aleatórios.

Capítulos 6–8 – Dados Compostos:

- Arrays uni e bidimensionais.
- Ponteiros: o coração de C.
- Strings e manipulação de caracteres.

Capítulo 9 – E/S Refinada:

- Domínio completo de `printf` e `scanf`.

Capítulos 10–12 – Estruturas e Persistência:

- `struct`, `union`, manipulação de bits.
- Arquivos sequenciais e de acesso aleatório.
- Listas, pilhas, filas, árvores – dinâmicas e poderosas.

Você é agora capaz de:

- Modelar problemas complexos com tipos próprios.
- Implementar estruturas de dados clássicas do zero.
- Persistir informações em arquivos.
- Construir programas significativos: jogos, simulações, calculadoras, analisadores de texto, e até compiladores.

O que ainda falta? Os capítulos 13–14 do livro abordam:

- **Cap. 13:** pré-processador (`#define`, `#ifdef`, macros).
- **Cap. 14:** tópicos avançados (`va_list`, manipulação de sinais, alocação dinâmica avançada, processamento de linha de comando).

Mas, em essência, **você já domina C**. Está pronto para:

- Ler código-fonte do Linux, do Git, do Vim.
- Contribuir com projetos open-source escritos em C.
- Aprender outras linguagens (C++, Rust, Go) com facilidade – elas herdam grande parte da sintaxe e dos conceitos de C.
- Trabalhar com sistemas embarcados, drivers de dispositivos, kernels.

Estruturas de dados: o ponto de virada da carreira

Os conceitos do Capítulo 12 não são apenas mais um tópico – são um **ponto de virada** na sua maturidade técnica:

Antes: “como faço para resolver este problema específico?”

Depois: “qual estrutura de dados é mais adequada para esta família de problemas?”

Onde estruturas de dados aparecem na carreira:

- **Entrevistas de emprego:** 80% das perguntas em FAANG envolvem listas, árvores, hashes, grafos.
- **Code review:** “por que você usou array aqui em vez de hash table?”
- **Performance:** a diferença entre $O(n)$ e $O(\log n)$ pode ser a diferença entre uma aplicação que funciona e outra que falha sob carga.
- **Design de sistemas:** bancos de dados usam B-trees; caches usam hash tables; redes usam filas; UNDO usa pilhas.

Próximos níveis (autoestudo):

- **Estruturas avançadas:** grafos, heaps, tries, segment trees.
- **Análise de algoritmos:** notação Big-O, master theorem, amortização.
- **Estruturas balanceadas:** AVL, Red-Black, B-trees.
- **Persistência:** estruturas funcionais imutáveis.
- **Concorrência:** estruturas lock-free, CAS, transactional memory.

Livros recomendados:

- Cormen et al., *Introduction to Algorithms* (CLRS).
- Sedgewick, *Algorithms in C*.
- Knuth, *The Art of Computer Programming* (vol. 1, 3).

Boa prática de programação**Conselho final do Cap. 12:**

A programação é, no fundo, sobre *estruturar informação*. Tipos primitivos (`int`, `double`) representam fatos isolados; arrays representam coleções homogêneas; `struct` representa entidades do mundo real; listas e árvores representam relações entre essas entidades.

Quando você domina essa hierarquia – de bits a estruturas dinâmicas – você pode modelar *qualquer coisa*. Os 12 capítulos da progressão Deitel construíram essa hierarquia passo a passo, sem atalhos.

O segredo da Programação (com P maiúsculo): não é decorar sintaxe. É treinar a mente para reconhecer padrões, dividir problemas, escolher representações adequadas, e expressar soluções em código limpo e correto. Você está pronto para o próximo nível. Bons estudos!